TASK 3: TASK AUTOMATION WITH PYTHON SCRIPTS

Complete Python Mini-Project

Goal: Automate a small, real-life repetitive task using a Python script.

Selected idea: Extract all email addresses from a text file and save them into another file.

1. Objective

This program reads a .txt file, finds email addresses using Python's regular-expression module (re), removes duplicates while keeping the original order, and writes the results to a new text file.

2. Python Source Code

```
import re
from pathlib import Path

# Input and output file names
input_file = Path("input.txt")
output_file = Path("emails_found.txt")

try:
    # Read all text from the input file
    text = input_file.read_text(encoding="utf-8")

    # Find email addresses in the text
    pattern = r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}"
    emails = re.findall(pattern, text)

    # Remove duplicates while keeping the original order
    unique_emails = list(dict.fromkeys(emails))

    # Save the email addresses to a new file
    output_file.write_text(
        "\n".join(unique_emails) if unique_emails else "No email addresses found.",
        encoding="utf-8"
    )

    print(f"Found {len(unique_emails)} unique email address(es).")
    print(f"Results saved to: {output_file}")

except FileNotFoundError:
    print("Error: input.txt was not found.")
    print("Create input.txt in the same folder as this Python script.")
except OSError as error:
    print("A file error occurred:", error)
```

3. Sample Input File (input.txt)

```
Contact us at support@example.com or keerthana@example.org.\nFor help, write to support@example.com.
```

4. Sample Output File (emails_found.txt)

```
support@example.com\nkeerthana@example.org
```

5. How to Run the Program

1. Install Python 3 if it is not already installed.
2. Create a folder for the project.
3. Save the program as **extract_emails.py** inside that folder.
4. Create a text file named **input.txt** in the same folder and add some sample text containing email addresses.
5. Open a terminal or command prompt in the project folder.
6. Run the command: `python extract_emails.py` (on some systems, use `python3 extract_emails.py`).
7. The program creates **emails_found.txt** containing the unique email addresses.

6. Key Concepts Used

Concept	Purpose
re module	Uses a regular expression to find email-like patterns in text.
Path / file handling	Reads input.txt and writes emails_found.txt.
read_text()	Reads the contents of the input file.
write_text()	Writes extracted addresses to the output file.
try-except	Handles a missing input file and other file errors.
list / dict.fromkeys()	Removes duplicate email addresses while preserving order.

7. Algorithm

1. Start the program.
2. Open and read input.txt.
3. Search the text for email addresses using a regular expression.
4. Remove duplicate addresses while preserving their order.
5. Write the results to emails_found.txt.
6. Display a success message; if a file error occurs, display an error message.
7. Stop the program.

8. Conclusion

This mini-project demonstrates how Python can automate repetitive file-processing work. It uses regular expressions, file handling, lists, and exception handling to extract email addresses and save them automatically. Use only text files and email addresses you are authorized to process.